Logic Solver AI: An Intelligent Decision System

A Full-Stack Constraint-Logic Framework for Automated Problem Solving

Project Overview

What is the Logic Solver AI?

- **Core Concept:** A web-based AI system processing user-defined constraints to generate mathematically proven solutions.
- **The Hybrid Architecture:** Integration of modern web (React/Node.js) with classical AI logic (SWI-Prolog).
- **Primary Capabilities:**
  - Solving complex constraint satisfaction problems.
  - Evaluating discrete mathematics (Propositional Logic, Graph Theory).
  - Providing rule-based decision-making and scoring.
- **Goal:** Convert raw input into intelligent decisions backed by step-by-step logical proofs.

Problem Statement

- **Static Limitations:** Traditional software relies on predefined if-else rules and cannot perform dynamic logical inference.
- **Manual Inefficiency:** Solving logic proofs or network graphs manually is tedious and highly prone to human error.
- **The "Black Box" Problem:** Modern Machine Learning acts as an opaque system; academic and analytical fields require **Explainable AI**.

> **Our Solution:** A system that applies strict mathematical constraints and outputs completely consistent, proven results.

System Objectives

- **Automate Logical Inference:** Derive new facts from user-provided premises.
- **Constraint Programming:** Utilize specialized solvers to filter out impossible solutions dynamically.
- **Bridge Web and AI:** Create a seamless API pipeline between browser-based frontend and localized SWI-Prolog.
- **Transparent Proofs:** Output the final answer along with the step-by-step logic path.
- **Data Integrity:** Rigorous input validation before passing to the AI engine.

System Architecture

Frontend (React + Vite)

- Captures logic variables & renders complex outputs visually.

Backend (Node.js + Express)

- Orchestration bridge; manages Prolog child processes.

Database (MongoDB)

- Stores user profiles and historical query results.

Solver (SWI-Prolog)

- The AI brain containing the knowledge base and logic rules.

End-to-End Workflow

- **Step 1:** User inputs logic rules into the React interface.
- **Step 2:** Node.js sanitizes data and checks for missing values.
- **Step 3:** Backend translates JSON into Prolog-compatible syntax.
- **Step 4:** Data injected into SWI-Prolog execution environment.
- **Step 5:** Engine applies knowledge base rules and CLP(FD) solving.
- **Step 6:** Outcome returned, saved to MongoDB, and rendered.

Core AI Engine: SWI-Prolog

Why SWI-Prolog? Perfect for symbolic reasoning and discrete math.

- **CLP(FD):** Constraint Logic Programming over Finite Domains helps solve combinatorial problems by shrinking the search space.
- **Dynamic Assertions:** Injects user inputs at runtime rather than using hardcoded data.
- **Declarative Nature:** Focused on *what* the problem is rather than *how* to compute the result.

Key Feature: Propositional Logic

Automating Discrete Mathematics:

- **KB Processing:** Users input premises like (A v ~C) -> B.
- **Goal Evaluation:** Prove targets as mathematically true or false.
- **Automated Computations:** Generates Truth Tables and checks for Logical Equivalence.
- **Explainability:** Displays the exact mathematical proof line-by-line.

Key Feature: Graph Theory

Dynamic Construction

- Accepts edge relationships (e.g., A-B, B-C).

Node Traversal

- Executes pathfinding (BFS).

Relational Logic

- Efficiently detects isolated components and optimal paths.

Visual Output

- Returns ordered traversal arrays and queue steps.

Data Transformation Bridge

Bridging Web with Classical AI:

- **Translation:** Web frameworks "speak" JSON; AI engines require logical predicates.
- **Backend Mapping:** Node.js writes dynamic facts like `assertz(skill(math))` based on requests.
- **Validation:** Rejects malformed formulas before invoking the engine, preventing Prolog syntax crashes.

Constraint & Scoring Strategy

- **Filtering via Constraints:**
  - Domain constraints limit variables.
  - Conditional constraints prevent conflicting rules.
- **Weighted Evaluation:** Each verified variable contributes to a cumulative score.
- **Ranking:** Mathematically valid outcomes are ranked to select the absolute best match.

Testing & Results

- **Test Scenarios:** Successfully mapped complex inputs to generate correct proofs and optimal paths.
- **Performance:** Millisecond response times for complex proofs.
- **Reliability:** 100% deterministic consistency (identical inputs yield identical results).
- **Error Handling:** API validation intercepting bad data types without crashing the backend.

Technology Stack

React + Vite | Node.js + Express | MongoDB + Mongoose | SWI-Prolog | CLP(FD) | REST API

> A robust stack combining high-performance UI state management with industrial-strength logic reasoning.

Learning Outcomes

- **MERN + Prolog Integration:** Deep practical knowledge of bridging web stacks with logic programming.
- **REST API Engineering:** Managing asynchronous web servers with synchronous AI executables.
- **Constraint Models:** Understanding declarative AI models over traditional imperative programming.
- **Validation Pipelines:** Engineering robust data protection for engine stability.

Future Enhancements

- **Advanced Algebra:** Expand knowledge base for complex algebraic constraints.
- **Hybrid AI:** Integrate Machine Learning for "fuzzy" inputs alongside strict logic.
- **Optimization:** Database indexing for faster historical query retrieval.

Conclusion: Successfully built a transparent, intelligent decision system with flawless mathematical accuracy.

Q & A

Thank You!

Questions regarding system architecture or logic implementation are welcome.